

Task 1 — Install Jenkins and run Jenkins on port 8081

1. Objective

The objective of this task was to:

- Launch an Amazon Linux 2023 EC2 instance.
- Connect to the EC2 instance using SSH.
- Verify the operating system and system architecture.
- Install Java 21, which is required to run Jenkins.
- Install Jenkins LTS using the official Jenkins RPM repository.
- Configure Jenkins to run on port 8081 instead of the default port 8080.
- Start and verify the Jenkins service.
- Access Jenkins through a web browser and complete the initial setup.
- Create and verify the Jenkins administrator user.

2. EC2 Instance Configuration

An EC2 instance was created for the Jenkins server.

Instance Configuration

Configuration	Value
Instance Name	Jenkins-01
AMI	Amazon Linux 2023
Instance Type	t3.micro
Key Pair	jenkins
Public IP	Enabled
Root Volume	20 GiB
Volume Type	gp3
Architecture	x86_64

Security Group Configuration

The following inbound rules were configured:

Type	Port	Source	Purpose
SSH	22	My IP	SSH access to EC2
Custom TCP	8081	My IP	Jenkins web access

Restricting both ports to **My IP** limits access to the Jenkins server from the configured client IP rather than allowing access from the entire internet.

3. Connect to the EC2 Instance

The EC2 instance was connected successfully using SSH.

The initial shell prompt was:

```
[ec2-user@ip-172-31-11-210 ~]$
```

The session was then switched to the root user using:

```
sudo su
```

The prompt changed to:

```
[root@ip-172-31-11-210 ec2-user]#
```

This allowed the installation and configuration commands to be executed with root privileges.

4. Verify Operating System

The Amazon Linux version was verified using:

```
cat /etc/os-release
```

The output confirmed:

```
NAME="Amazon Linux"
VERSION="2023"
```

```
ID="amzn"  
VERSION_ID="2023"  
PRETTY_NAME="Amazon Linux 2023.12.20260918"
```

Observation

The server is running **Amazon Linux 2023**, which is the operating system selected during EC2 creation.

5. Verify System Architecture

The system architecture was checked using:

```
uname -m
```

Output:

```
x86_64
```

Observation

The EC2 instance is using the x86_64 architecture.

6. Check Java Installation

Java was checked before installing Jenkins using:

```
java -version
```

Java was not installed on the newly launched server.

Since Jenkins requires Java to run, Java was installed before starting Jenkins.

7. Install Java 21

Java 21 was installed using Amazon Corretto:

```
dnf install java-21-amazon-corretto -y
```

After installation, Java was verified using:

```
java -version
```

Observation

Java 21 was successfully installed and made available to the system.

Java is required because Jenkins is a Java-based application and the Jenkins service uses the installed Java runtime to execute Jenkins.

8. Configure the Official Jenkins Repository

The official Jenkins RPM repository was configured so that Jenkins could be installed using the system package manager.

The Jenkins repository file was downloaded using:

```
wget -O /etc/yum.repos.d/jenkins.repo \  
https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

The Jenkins repository signing key was imported using:

```
rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
```

The package metadata was then refreshed using:

```
dnf upgrade -y
```

Observation

The Jenkins LTS RPM repository was successfully configured on the Amazon Linux server.

9. Install Jenkins

Jenkins was installed using:

```
dnf install jenkins -y
```

The installed Jenkins package was verified using:

```
rpm -q jenkins
```

The installed package was:

```
jenkins-2.568.3-1.noarch
```

Observation

Jenkins LTS was successfully installed on the EC2 instance.

10. Configure Jenkins Port

The requirement was to run Jenkins on port:

```
8081
```

instead of the default:

```
8080
```

The installed Jenkins version uses a **systemd service configuration**.

The service configuration was checked using:

```
systemctl cat jenkins
```

The service indicated that systemd manages Jenkins.

Instead of modifying the original service file under:

```
/usr/lib/systemd/system/jenkins.service
```

a systemd override configuration was created.

The override file was:

```
/etc/systemd/system/jenkins.service.d/override.conf
```

The following configuration was added:

```
[Service]
```

```
Environment="JENKINS_PORT=8081"
```

The configuration was verified using:

```
cat /etc/systemd/system/jenkins.service.d/override.conf
```

Output:

```
[Service]
```

```
Environment="JENKINS_PORT=8081"
```

Observation

The Jenkins systemd service was configured to use port 8081.

Using a systemd override is preferable to directly modifying the package-managed service file because package-managed files should not be manually modified.

11. Reload systemd Configuration

After creating the Jenkins service override, systemd was instructed to reload the configuration:

```
systemctl daemon-reload
```

Observation

systemd successfully reloaded the Jenkins service configuration, including the new port setting.

12. Start Jenkins Service

Jenkins was started using:

```
systemctl start jenkins
```

The service status was checked using:

```
systemctl status jenkins --no-pager
```

The Jenkins service showed:

Active: active (running)

Observation

The Jenkins service started successfully and was running on the EC2 instance.

13. Verify Jenkins Port

The listening port was checked using:

```
ss -lntp | grep 8081
```

The command confirmed that Jenkins was listening on port:

8081

Observation

Jenkins was successfully bound to port 8081.

This also matched the EC2 Security Group rule that allowed inbound TCP traffic on port 8081 from the configured client IP.

14. Retrieve Initial Jenkins Administrator Password

During the first startup, Jenkins generated an initial administrator password.

The password was retrieved using:

```
cat /var/lib/jenkins/secrets/initialAdminPassword
```

The generated password was copied for use during the initial Jenkins setup.

Observation

The initial administrator password was successfully retrieved from the Jenkins home directory.

15. Access Jenkins Through Browser

Jenkins was accessed using the EC2 instance's public IP address and port 8081:

<http://<EC2-Public-IP>:8081>

For example:

<http://<EC2-Public-IP>:8081>

The Jenkins **Unlock Jenkins** page appeared.

The initial administrator password retrieved from:

`/var/lib/jenkins/secrets/initialAdminPassword`

was entered to unlock Jenkins.

16. Complete Initial Jenkins Setup

The initial Jenkins setup was completed through the Jenkins web interface.

A Jenkins administrator account was created.

The configured username was verified as:

asifshaik

The user was able to log in successfully to the Jenkins dashboard.

Observation

The Jenkins web interface was accessible and the administrator user was successfully created.

17. Validation

The following items were verified successfully:

Validation	Status
EC2 instance launched	Completed
Amazon Linux 2023 verified	Completed
x86_64 architecture verified	Completed
Java 21 installed	Completed
Jenkins repository configured	Completed
Jenkins LTS installed	Completed
Jenkins port configured to 8081	Completed
systemd configuration reloaded	Completed
Jenkins service started	Completed
Jenkins service running	Completed
Port 8081 verified	Completed
Initial Jenkins password retrieved	Completed
Jenkins web interface accessed	Completed
Administrator user created	Completed
asifshaik login verified	Completed

18. Architecture / Flow

The implementation flow can be represented as:

User Laptop

|

| SSH :22

↓

Amazon Linux 2023 EC2

|

| Java 21

↓

Jenkins LTS

|

| HTTP :8081

↓

Jenkins Web Interface

|

↓

Administrator User
(asifshaik)

The network access flow is:

Browser



| TCP 8081



EC2 Security Group



| Allowed from My IP



EC2 Instance



Jenkins



| Port 8081



Jenkins Dashboard

The screenshot displays the AWS Management Console interface for an EC2 instance. The left sidebar shows the navigation menu with categories like EC2, Images, and Elastic Block Store. The main content area is titled 'Instance summary for i-00b23df5c89a8ba1e (jenkins)' and includes various details about the instance's configuration and status.

Property	Value
Instance ID	i-00b23df5c89a8ba1e
Public IPv4 address	44.199.215.192 open address
Instance state	Running
Private IPv4 addresses	172.31.11.210
Public DNS	ec2-44-199-215-192.compute-1.amazonaws.com open address
IPv6 address	-
Instance type	t3.micro
Private IP DNS name (IPv4 only)	ip-172-31-11-210.ec2.internal
Subnet ID	subnet-061bd22cea53fcb99 open
Hostnames type	IP name: ip-172-31-11-210.ec2.internal
VPC ID	vpc-048b15dd0eb01d950 open
Answer private resource DNS name	-
Auto-assigned IP address	44.199.215.192 [Public IP]
Subnet ID	subnet-061bd22cea53fcb99 open
IAM role	-
Instance ARN	arn:aws:ec2:us-east-1:331116047691:instance/i-00b23df5c89a8ba1e
IMDSv2	Required
Elastic IP addresses	-
AWS Compute Optimizer finding	Opt-in to AWS Compute Optimizer for recommendation Learn more
Auto Scaling Group name	-
Managed	false


```
root@ip-172-31-11-210/home/ec2-user
[root@ip-172-31-11-210 ec2-user]# dnf install java-21-amazon-corretto -y
Amazon Linux 2023 repository
Amazon Linux 2023 kernel Livepatch repository
Dependencies resolved.
71 MB/s | 80 MB 00:01
724 kB/s | 82 kB 00:00

Package Architecture Version Repository Size
-----
Installing:
java-21-amazon-corretto x86_64 1:21.0.12+9-1.amzn2023.1 amazonlinux 217 k
Installing dependencies:
alsa-lib x86_64 1.2.7.2-1.amzn2023.0.4 amazonlinux 503 k
cairo x86_64 1.18.0-4.amzn2023.0.3 amazonlinux 717 k
dejavu-sans-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.3 M
dejavu-sans-mono-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 467 k
dejavu-serif-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.0 M
fontconfig x86_64 2.13.94-2.amzn2023.0.2 amazonlinux 273 k
fonts-filesystem noarch 1:2.0.5-12.amzn2023.0.2 amazonlinux 9.5 k
freetype x86_64 2.13.2-5.amzn2023.0.2 amazonlinux 422 k
glib x86_64 2.2.1-9.amzn2023.0.4 amazonlinux 49 k
google-noto-fonts-common noarch 20240401-1.amzn2023.0.2 amazonlinux 17 k
google-noto-sans-vf-fonts noarch 20240401-1.amzn2023.0.2 amazonlinux 593 k
graphite2 x86_64 1.3.14-7.amzn2023.0.3 amazonlinux 97 k
harfbuzz x86_64 7.0.0-2.amzn2023.0.2 amazonlinux 873 k
java-21-amazon-corretto-headless x86_64 1:21.0.12+9-1.amzn2023.1 amazonlinux 96 M
javapackages-filesystem noarch 6.0.0-7.amzn2023.0.6 amazonlinux 12 k
langpacks-core-font-en x86_64 1.0-21.amzn2023.0.4 amazonlinux 10 k
libICE x86_64 1.1.1-3.amzn2023.0.1 amazonlinux 76 k
libSM x86_64 1.2.4-3.amzn2023.0.1 amazonlinux 45 k
libX11 x86_64 1.8.10-2.amzn2023.0.1 amazonlinux 659 k
libX11-common noarch 1.8.10-2.amzn2023.0.1 amazonlinux 147 k
libXau x86_64 1.0.11-6.amzn2023.0.1 amazonlinux 33 k
libXext x86_64 1.3.6-1.amzn2023.0.1 amazonlinux 42 k
libXft x86_64 1.8.2-1.amzn2023.0.1 amazonlinux 42 k
libXinerama x86_64 1.1.5-6.amzn2023.0.1 amazonlinux 16 k
libXrandr x86_64 1.5.4-3.amzn2023.0.1 amazonlinux 29 k
libXrender x86_64 0.9.11-6.amzn2023.0.1 amazonlinux 29 k
libXt x86_64 1.3.0-3.amzn2023.0.1 amazonlinux 183 k
libXtst x86_64 1.2.5-1.amzn2023.0.1 amazonlinux 29 k
libbrotli x86_64 1.0.9-4.amzn2023.0.2 amazonlinux 315 k
libjpeg-turbo x86_64 2.1.4-2.amzn2023.0.5 amazonlinux 190 k
libpng x86_64 2:1.6.37-10.amzn2023.0.13 amazonlinux 128 k
libxkb x86_64 1.17.0-1.amzn2023.0.1 amazonlinux 235 k
pixman x86_64 0.43.4-1.amzn2023.0.4 amazonlinux 296 k
xml-common noarch 0.6.3-56.amzn2023.0.2 amazonlinux 32 k

Transaction Summary
-----
Install 35 Packages

Total download size: 105 M
Installed size: 283 M
Downloading Packages:
(1/35): alsa-lib-1.2.7.2-1.amzn2023.0.4.x86_64.rpm 15 MB/s | 503 kB 00:00
(2/35): cairo-1.18.0-4.amzn2023.0.3.x86_64.rpm 18 MB/s | 717 kB 00:00
(3/35): dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch.rpm 27 MB/s | 1.3 MB 00:00
(4/35): dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch.rpm 16 MB/s | 467 kB 00:00
(5/35): dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch.rpm 32 MB/s | 1.0 MB 00:00
(6/35): fontconfig-2.13.94-2.amzn2023.0.2.x86_64.rpm 7.0 MB/s | 273 kB 00:00
```

```
root@ip-172-31-11-210/home/ec2-user
[root@ip-172-31-11-210 ec2-user]# java -version
openjdk version "21.0.12-LTS" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12+9-1 (build 21.0.12.149-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12+9-1 (build 21.0.12.149-LTS, mixed mode, sharing)
[root@ip-172-31-11-210 ec2-user]# which java
/usr/bin/java
[root@ip-172-31-11-210 ec2-user]# wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo
2026-09-28 10:06:15 -- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.131, 2a04:ee27:79::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)[146.75.38.131]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo 100%[=====] 267 --.-KB/s in 0s

2026-09-28 10:06:15 (20.7 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]

[root@ip-172-31-11-210 ec2-user]# rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
[root@ip-172-31-11-210 ec2-user]# dnf upgrade -y
Jenkins-stable
Importing GPG key 0x14ABFC68:
Userid : "Jenkins Project <jenkinsci-board@googlegroups.com>"
Fingerprint: 5E38 6C40 855F 0150 4CAE 8ACF 7198 F487 144B FC68
From : https://pkg.jenkins.io/rpm-stable/repo/data/repoond.xml.key
Jenkins-stable
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-11-210 ec2-user]# dnf install jenkins -y
Last metadata expiration check: 0:00:25 ago on Mon Sep 28 10:06:37 2026.
Dependencies resolved.

Package Architecture Version Repository Size
-----
Installing:
jenkins noarch 2.568.3-1 jenkins 96 M

Transaction Summary
-----
Install 1 Package

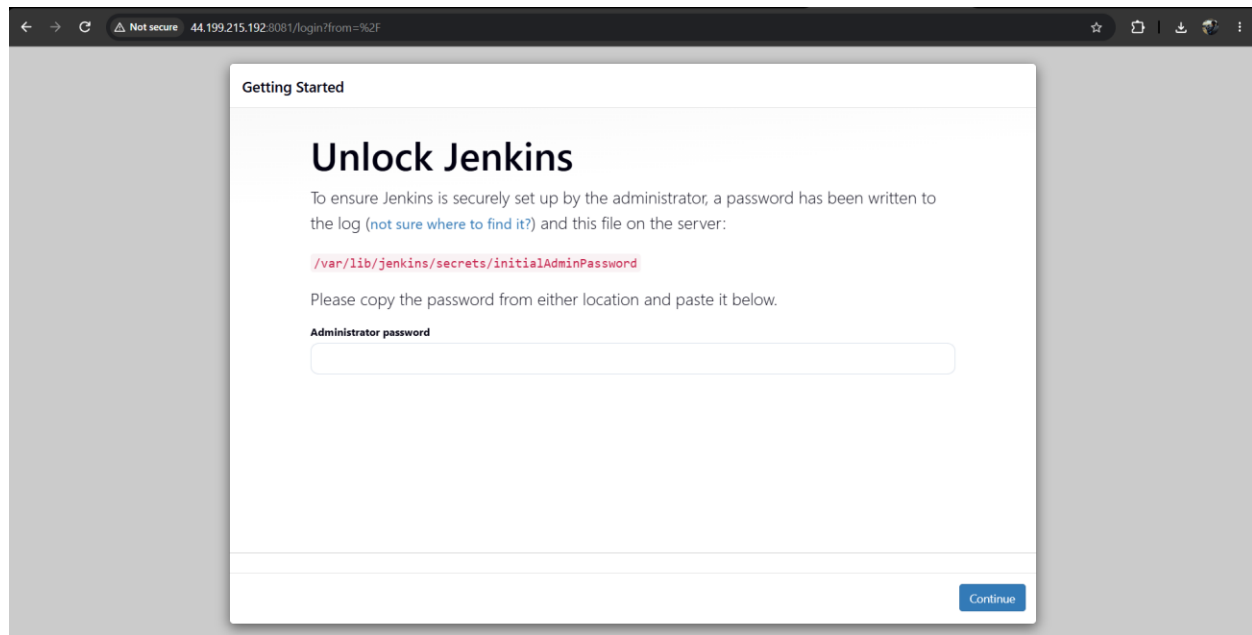
Total download size: 96 M
Installed size: 96 M
Downloading Packages:
jenkins-2.568.3-1.noarch.rpm 86 MB/s | 96 MB 00:01
Total 84 MB/s | 96 MB 00:01

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing:
Running scriptlet: jenkins-2.568.3-1.noarch 1/1
Installing: jenkins-2.568.3-1.noarch 1/1
```

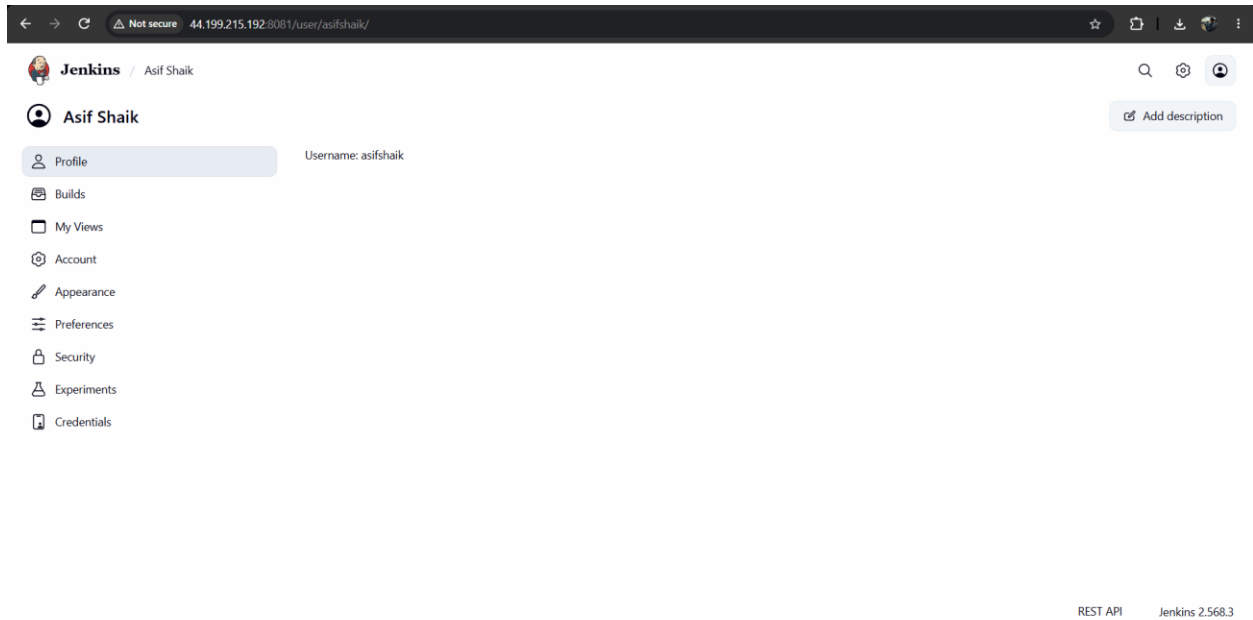
```
[root@ip-172-31-11-210 ec2-user]# cat /etc/systemd/system/jenkins.service.d/override.conf
[Service]
Environment="JENKINS_PORT=8081"
[root@ip-172-31-11-210 ec2-user]# systemctl daemon-reload
[root@ip-172-31-11-210 ec2-user]# systemctl start jenkins
[root@ip-172-31-11-210 ec2-user]# rpm -q jenkins
jenkins-2.568.3-1.noarch
[root@ip-172-31-11-210 ec2-user]# systemctl start jenkins
[root@ip-172-31-11-210 ec2-user]# systemctl status jenkins --no-pager
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; disabled; preset: disabled)
   Drop-In: /etc/systemd/system/jenkins.service.d
            -override.conf
   Active: active (running) since Mon 2026-09-28 10:16:29 UTC; 2min 2s ago
   Main PID: 26954 (java)
     Tasks: 27 (limit: 1059)
    Memory: 275.9M
       CPU: 20.780s
   CGroup: /system.slice/jenkins.service
           └─26954 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8081

Sep 28 10:16:24 ip-172-31-11-210.ec2.internal jenkins[26954]: [LF]>
Sep 28 10:16:24 ip-172-31-11-210.ec2.internal jenkins[26954]: [LF]> *****
Sep 28 10:16:24 ip-172-31-11-210.ec2.internal jenkins[26954]: [LF]> *****
Sep 28 10:16:24 ip-172-31-11-210.ec2.internal jenkins[26954]: [LF]> *****
Sep 28 10:16:29 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:16:29.415+0000 [id=31] INFO Jenkins.TailReactorRunner$1#onAttained: Completed initialization
Sep 28 10:16:29 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:16:29.441+0000 [id=24] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
Sep 28 10:16:29 ip-172-31-11-210.ec2.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Sep 28 10:16:29 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:16:29.732+0000 [id=50] INFO h.m.DownloadService$Downloadable#load: Obtained the updated data file for hudson.maveninstaller
Sep 28 10:16:29 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:16:29.734+0000 [id=50] INFO hudson.util.Retrier#start: Performed the action check updates server success. the attempt #1
Sep 28 10:16:34 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:16:34.512+0000 [id=70] WARNING h.n.DiskSpaceMonitorDescriptor#markNodeOfflineOnline: Making Built-In N.k of disk space
Hint: Some lines were ellipsized, use -l to show in full.
[root@ip-172-31-11-210 ec2-user]# ss -ltp | grep 8081
LISTEN 0          50          *:8081          *:8081          users:((("java".pid=26954,fd=9))

```



```
[root@ip-172-31-11-210 ec2-user]# cat /var/lib/jenkins/secrets/initialAdminPassword
3e88e6986b2b434dabdc314361716924
```



19. Result

Jenkins LTS was successfully installed on an **Amazon Linux 2023 EC2 instance**.

Java 21 was installed as the Jenkins runtime. Jenkins was configured through a systemd override to listen on **port 8081**. The Jenkins service was started successfully and verified to be running.

The Jenkins web interface was accessed using the EC2 public IP and port 8081, and the administrator account **asifshaik** was successfully created and verified.

20. Conclusion

The Jenkins installation task was completed successfully.

The EC2 instance is now functioning as a Jenkins server, with Jenkins LTS running on **port 8081**. The required operating system, Java runtime, Jenkins package, systemd configuration, service, network access, and initial administrator setup were successfully configured and validated.

Task 2 — Secure Jenkins Server

Objective

The objective of this task was to secure the Jenkins server by configuring authentication, disabling anonymous access, enabling CSRF protection, controlling user registration, configuring API token settings, and ensuring that the Jenkins service starts automatically after a system reboot.

Step 1 — Open Jenkins Security Configuration

After successfully installing Jenkins and accessing Jenkins through:

<http://<EC2-Public-IP>:8081>

I logged in using the Jenkins administrator account.

Then I navigated to:

Manage Jenkins → Security

The Jenkins Security configuration page was opened.

Observation

The Security page contained configuration options for:

- Authentication
- Authorization
- CSRF Protection
- API Tokens
- Agents
- Git Host Key Verification
- Markup Formatter
- Security warnings
- Other Jenkins security-related settings

Step 2 — Configure Jenkins Security Realm

Under **Authentication**, I selected:

Security Realm → Jenkins' own user database

This allows Jenkins to manage its own users and passwords instead of depending on an external authentication system.

Configuration

- Security Realm: **Jenkins' own user database**
- Allow users to sign up: **Disabled / Unchecked**

Observation

Only users created by the Jenkins administrator can have Jenkins accounts.

Public users cannot create their own Jenkins accounts.

Why this was configured

Allowing unrestricted user registration could allow unauthorized users to create accounts on the Jenkins server. Disabling self-registration ensures that user creation remains under administrator control.

Step 3 — Configure Authorization

Initially, Jenkins was configured with:

Logged-in users can do anything

For the current security task, this ensures that authentication is required before accessing Jenkins.

The anonymous read access option was kept disabled.

Configuration

- Authorization: **Logged-in users can do anything**
- Allow anonymous read access: **Disabled / Unchecked**

Observation

Users must authenticate with Jenkins before accessing protected Jenkins functionality.

Anonymous users are not given read access to Jenkins.

Note: Limited permissions for the DevOps and Testing users will be configured in the next task using Jenkins authorization controls.

Step 4 — Configure Markup Formatter

Under **Markup Formatter**, I kept:

Plain text

Observation

Jenkins will treat user-provided descriptions and similar content as plain text rather than allowing arbitrary HTML markup.

This reduces the possibility of unsafe HTML content being rendered inside Jenkins.

Step 5 — Verify CSRF Protection

Under **CSRF Protection**, the default configuration was kept enabled:

Default Crumb Issuer

Observation

Jenkins uses a CSRF crumb to help protect requests against Cross-Site Request Forgery attacks.

The default Jenkins crumb issuer provides CSRF protection without requiring custom configuration.

Result

CSRF protection remained enabled.

Step 6 — Configure API Token Settings

The Security page contained API Token options such as:

- Generate a legacy API token for each newly created user
- Allow users to manually create a legacy API token
- Enable API Token usage statistics

The legacy API token options were **left disabled** because they were marked as **Not recommended**.

Observation

Jenkins will not automatically generate legacy API tokens for newly created users, and users are not given the old legacy-token creation option.

Step 7 — Configure Git Host Key Verification

Under:

Git Host Key Verification Configuration

I selected:

Accept first connection

Observation

When Jenkins connects to a Git SSH host for the first time, Jenkins can accept and remember the host key.

This configuration is useful for the lab environment while still providing host-key checking after the initial connection.

Step 8 — Review Other Security Settings

I reviewed the remaining security settings on the page, including:

- Content Security Policy

- Sandbox Configuration
- Git Hooks
- Agent TCP port
- Security warnings

No unnecessary changes were made to these settings.

Observation

Only the security settings required for the current Jenkins security task were modified. This avoids introducing configuration changes that are not required for the lab.

Step 9 — Save the Security Configuration

After completing the required security configuration, I clicked:

Save

Jenkins applied the new security settings.

Observation

The Jenkins server continued running successfully after the security configuration was saved.

Step 10 — Enable Jenkins Service at System Startup

After securing Jenkins, I configured the Jenkins system service to start automatically whenever the EC2 instance starts.

From the Jenkins server terminal, I executed:

```
systemctl enable jenkins
```

Observation

The Jenkins service was configured to start automatically during system boot.

Step 11 — Verify Jenkins Service

I verified the Jenkins service using:

```
systemctl status jenkins --no-pager
```

Observation

The Jenkins service was in an active/running state.

The service was operating normally after applying the security configuration.

Step 12 — Verify Jenkins Port 8081

I verified that Jenkins was still listening on port 8081:

```
ss -lntp | grep 8081
```

Observation

Jenkins was listening on:

8081

Therefore, the security configuration did not change the Jenkins application port.

Step 13 — Verify Automatic Startup Configuration

I checked whether Jenkins was enabled at boot:

```
systemctl is-enabled jenkins
```

Observation

The command returned:

enabled

Result

Jenkins is configured to automatically start when the EC2 instance is rebooted.

Step 14 — Test Anonymous Access

To verify that anonymous access was disabled, I opened Jenkins in an **Incognito/Private browser window** without using the existing authenticated browser session.

I accessed:

<http://<EC2-Public-IP>:8081>

Observation

Instead of directly displaying the Jenkins dashboard, Jenkins displayed the **Sign in** page.

Result

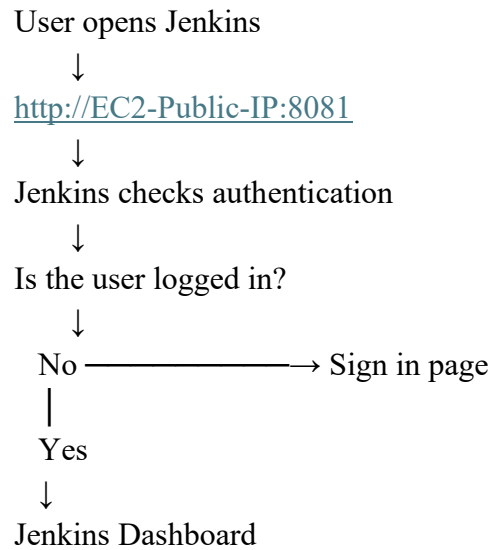
This confirmed that unauthenticated/anonymous users cannot directly access the Jenkins dashboard.

Security Configuration Summary

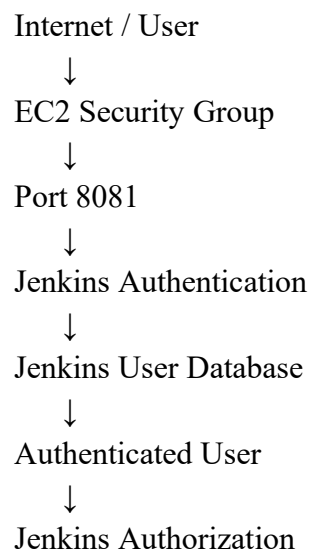
Security Setting	Configuration
Security Realm	Jenkins' own user database
User self-registration	Disabled
Authorization	Logged-in users can do anything
Anonymous read access	Disabled
Markup Formatter	Plain text
CSRF Protection	Default Crumb Issuer
Legacy API token generation	Disabled
Legacy API token creation	Disabled
Git Host Key Verification	Accept first connection
Jenkins service startup	Enabled
Jenkins service status	Active/Running

Jenkins application port	8081
Anonymous access test	Redirected to Sign in

Verification Flow



Overall Security Flow



← → ↺ ⚠ Not secure 44.199.215.192:8081/manage/configureSecurity/ ☆ 📄 🗑 🔄 👤

 Jenkins / Manage Jenkins ▾ / Security 🔍 ⚙ 👤

Security

Secure Jenkins; define who is allowed to access/use the system.

Authentication

☐ Disable "Keep me signed in" ?

Security Realm

Jenkins' own user database ▾ ?

☐ Allow users to sign up ?

Authorization

Logged-in users can do anything ▾ ?

☐ Allow anonymous read access ?

Markup Formatter

Save

Apply

```
[root@ip-172-31-11-210 ec2-user]# systemctl enable jenkins
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service - /usr/lib/systemd/system/jenkins.service.
[root@ip-172-31-11-210 ec2-user]# systemctl is-enabled jenkins
enabled
[root@ip-172-31-11-210 ec2-user]# systemctl status jenkins --no-pager
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Drop-In: /etc/systemd/system/jenkins.service.d
            ↳ override.conf
   Active: active (running) since Mon 2026-09-28 10:16:29 UTC; 1h 1min ago
   Main PID: 26954 (java)
     Tasks: 27 (limit: 1059)
    Memory: 569.0M
       CPU: 1min 10.816s
   CGroup: /system.slice/jenkins.service
           └─26954 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8081

Sep 28 10:26:22 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:22.503+0000 [id=228] INFO jenkins.InitReactorRunner$1onAttained: Listed all plugins
Sep 28 10:26:22 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:22.506+0000 [id=228] INFO jenkins.InitReactorRunner$1onAttained: Prepared all plugins
Sep 28 10:26:22 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:22.538+0000 [id=227] INFO jenkins.InitReactorRunner$1onAttained: Started all plugins
Sep 28 10:26:22 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:22.540+0000 [id=227] INFO jenkins.InitReactorRunner$1onAttained: Augmented all extensions
Sep 28 10:26:22 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:22.581+0000 [id=228] INFO jenkins.InitReactorRunner$1onAttained: System config loaded
Sep 28 10:26:22 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:22.591+0000 [id=227] INFO jenkins.InitReactorRunner$1onAttained: System config adapted
Sep 28 10:26:22 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:22.592+0000 [id=227] INFO jenkins.InitReactorRunner$1onAttained: Loaded all jobs
Sep 28 10:26:22 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:22.593+0000 [id=228] INFO jenkins.InitReactorRunner$1onAttained: Configuration for all jobs updated
Sep 28 10:26:23 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:23.095+0000 [id=227] INFO jenkins.InitReactorRunner$1onAttained: Completed initialization
Sep 28 10:26:23 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 10:26:23.096+0000 [id=80] INFO h.m.UpdateCenter$CompleteBatchJob#run: Completed installation of 92 plugins in 16 sec
[root@ip-172-31-11-210 ec2-user]# ss -ltnp | grep 8081
LISTEN 0      50          *:*                *:*                users:(("java",pid=26954,fd=9))
```

← → ↺ ⚠ Not secure 44.199.215.192:8081/manage/configureSecurity/ ☆ 📄 🗑 🔄 👤

 Jenkins / Manage Jenkins ▾ / Security 🔍 ⚙ 👤

This section allows you to suppress warnings, applicable to your Jenkins configuration, provided by the update center. If you do, they won't be shown by the update site warnings administration monitor. Checked warnings are reported (the default), unchecked warnings are hidden.

Security warnings ?

API Token

☐ Generate a legacy API token for each newly created user (Not recommended) ?

☐ Allow users to manually create a legacy API token (Not recommended) ?

☒ Enable API Token usage statistics ?

Content Security Policy

Content-Security-Policy enforcement is currently disabled. [Learn more.](#)

Git Host Key Verification Configuration

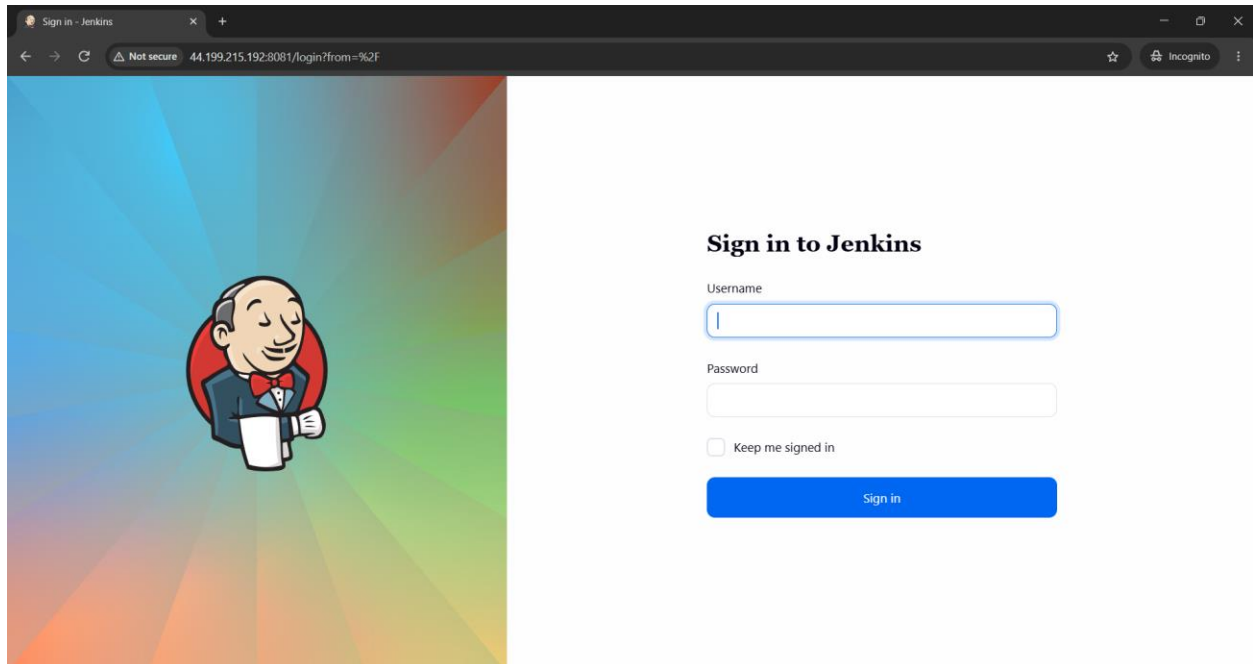
Host Key Verification Strategy ?

Accept first connection ▾

Save

Apply

44.199.215.192:8081



Result

The Jenkins server was successfully secured by configuring Jenkins' own user database, disabling user self-registration, disabling anonymous read access, keeping CSRF protection enabled, and disabling the legacy API-token options.

The Jenkins service was also configured to start automatically at system boot. The service remained active and Jenkins continued to listen on port 8081.

Anonymous access was tested through an Incognito browser, and Jenkins displayed the **Sign in** page, confirming that authentication is required.

Conclusion

Jenkins-01 Task 2 — **Secure Jenkins Server** was successfully completed.

The Jenkins server now requires users to authenticate before accessing Jenkins, anonymous access is disabled, CSRF protection is enabled, and the Jenkins service is configured for automatic startup.

The server was also verified after configuration to ensure that Jenkins remained active and accessible on port 8081.

The next task is to create the **DevOps** and **Testing** Jenkins users and configure them with **limited permissions** rather than administrator-level access.

Task 3 — Create users called DevOps, Testing in Jenkins with Limited access

Objective

The objective of this task was to create two Jenkins users, **DevOps** and **Testing**, and provide them with limited permissions.

The administrator account **asifshaik** was retained with full administrative access. Matrix-based security was used to control permissions for each Jenkins user.

Step 1 — Open Jenkins Security Configuration

I logged in to Jenkins using the administrator account:

Username: asifshaik

Then I navigated to:

Manage Jenkins → Security

Under the **Authorization** section, I selected:

Matrix-based security

Observation

After selecting Matrix-based security, Jenkins displayed a permissions matrix containing different categories such as:

- Overall
- Credentials
- Agent
- Job
- Run
- View
- SCM

The matrix allows permissions to be assigned individually to different users.

Step 2 — Add Administrator User to the Matrix

The asifshaik user was added to the authorization matrix.

Configuration

For the asifshaik user, I selected:

Overall → Administer 

Observation

The asifshaik account retained administrator-level access to Jenkins.

Important Point

The administrator permission was configured before saving the Matrix-based security configuration.

This was important because removing administrator access from the current account could result in losing access to Jenkins administration.

Step 3 — Create the DevOps User

I navigated to:

Manage Jenkins → Users → Create User

I created a new Jenkins user with:

- **Username:** DevOps
- **Password:** Configured during user creation
- **Full Name:** DevOps

The user was successfully created.

Observation

The DevOps user appeared as a separate Jenkins user and could be assigned permissions independently from the administrator account.

Step 4 — Create the Testing User

I again navigated to:

Manage Jenkins → Users → Create User

I created another Jenkins user with:

- **Username:** Testing
- **Password:** Configured during user creation
- **Full Name:** Testing

Observation

The Testing user was successfully created.

At this point, Jenkins contained three users:

Username	Purpose
asifshaik	Administrator
DevOps	Limited-access user
Testing	Limited-access user

Step 5 — Add DevOps to the Authorization Matrix

I returned to:

Manage Jenkins → Security → Authorization → Matrix-based security

I clicked:

Add user

and added:

DevOps

The DevOps user was then displayed as a row in the permissions matrix.

Step 6 — Configure DevOps Limited Permissions

For the DevOps user, I selected only the required limited permissions.

Overall

- Read ☒
- Administer ☐
- Manage ☐

Job

- Read ☒
- Build ☒

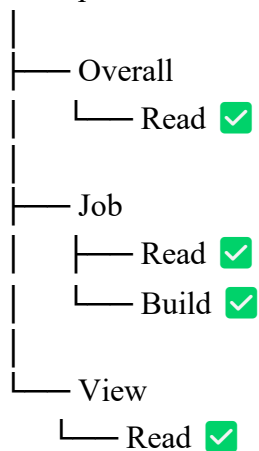
View

- Read ☒

All other unnecessary permissions were left unchecked.

DevOps Permission Summary

DevOps



Observation

The DevOps user could access Jenkins as an authenticated user but did not receive administrative permissions.

Step 7 — Add Testing to the Authorization Matrix

I clicked:

Add user

and added:

Testing

The Testing user appeared as another row in the permissions matrix.

Step 8 — Configure Testing Limited Permissions

For the Testing user, I selected the same limited permissions.

Overall

- Read ☒
- Administer ☐
- Manage ☐

Job

- Read ☒
- Build ☒

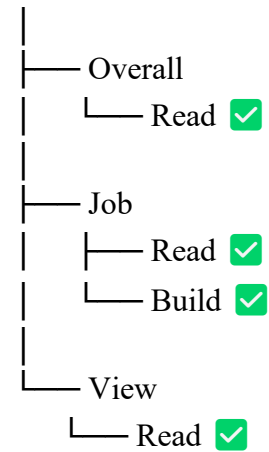
View

- Read ☒

All other unnecessary permissions were left unchecked.

Testing Permission Summary

Testing



Observation

The Testing user was given enough access to log in, view Jenkins jobs, and build jobs, without receiving Jenkins administration privileges.

Step 9 — Verify Administrator Permissions

Before saving the configuration, I verified the administrator row.

For:

asifshaik

the following permission was enabled:

Overall → Administer ✓

Observation

The administrator account retained administrative access.

This ensured that the current administrator would not be locked out after enabling Matrix-based security.

Step 10 — Save the Authorization Configuration

After verifying all three users and their permissions, I clicked:

Save

Jenkins applied the Matrix-based authorization configuration.

Observation

The security configuration was successfully saved, and Jenkins remained accessible.

Step 11 — Verify Administrator Access

I remained logged in as:

asifshaik

I verified that **Manage Jenkins** was still accessible.

Observation

The administrator account could access **Manage Jenkins**.

Result

This confirmed that the administrator permission was configured correctly.

Step 12 — Test DevOps User

I opened Jenkins in an **Incognito/Private browser window** and logged in using:

Username: DevOps

Observation

The DevOps user successfully logged in to Jenkins.

However, **Manage Jenkins was not available** to the DevOps user.

Result

This confirmed that DevOps did not have administrator-level access.

The Matrix-based authorization was successfully restricting the user.

Step 13 — Test Testing User

I logged out of the DevOps account and logged in using:

Username: Testing

Observation

The Testing user successfully logged in to Jenkins.

The **Manage Jenkins** administration option was not available.

Result

This confirmed that the Testing account was also restricted and did not have administrator privileges.

Step 14 — Permission Verification

The final permission structure was:

User	Overall Read	Overall Administer	Job Read	Job Build	View Read
asifshaik	—	✓	—	—	—
DevOps	✓	✗	✓	✓	✓
Testing	✓	✗	✓	✓	✓

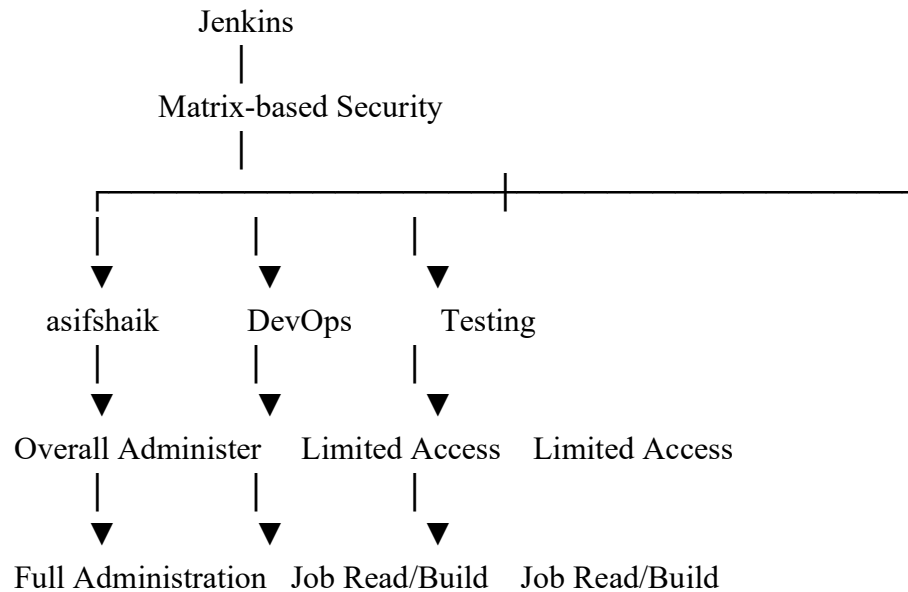
Access Verification

Test	asifshaik	DevOps	Testing
Login	✓	✓	✓
Jenkins dashboard	✓	✓	✓

Manage Jenkins	✓	✗	✗
Administrator access	✓	✗	✗

Authorization Flow

The final Jenkins authorization flow can be represented as:



Security Concept

Matrix-based security allows Jenkins permissions to be assigned individually to users.

Instead of giving every authenticated user complete access, permissions can be controlled according to the user's responsibilities.

For this task:

Administrator
 ↓
 asifshaik
 ↓
 Overall → Administer

while:

Limited Users



DevOps / Testing

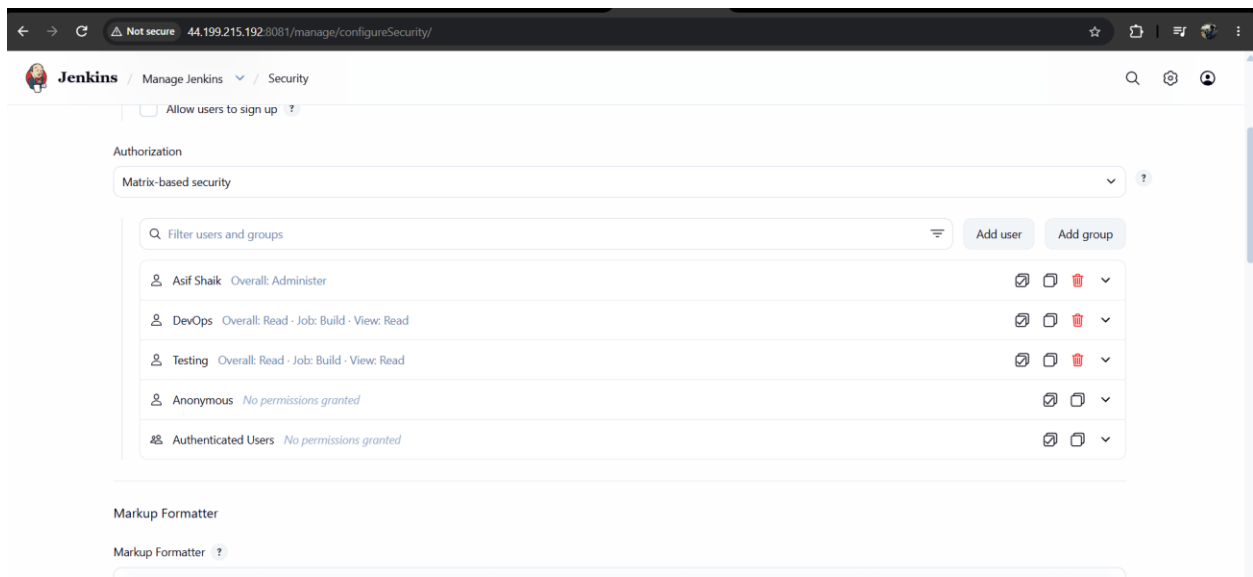
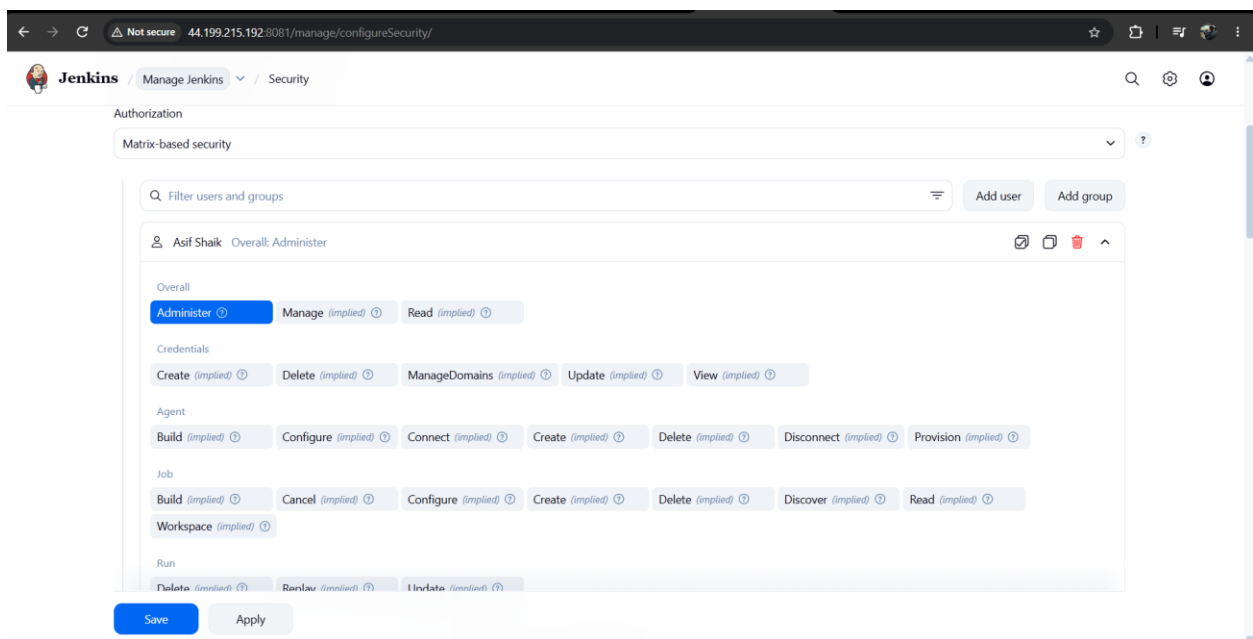


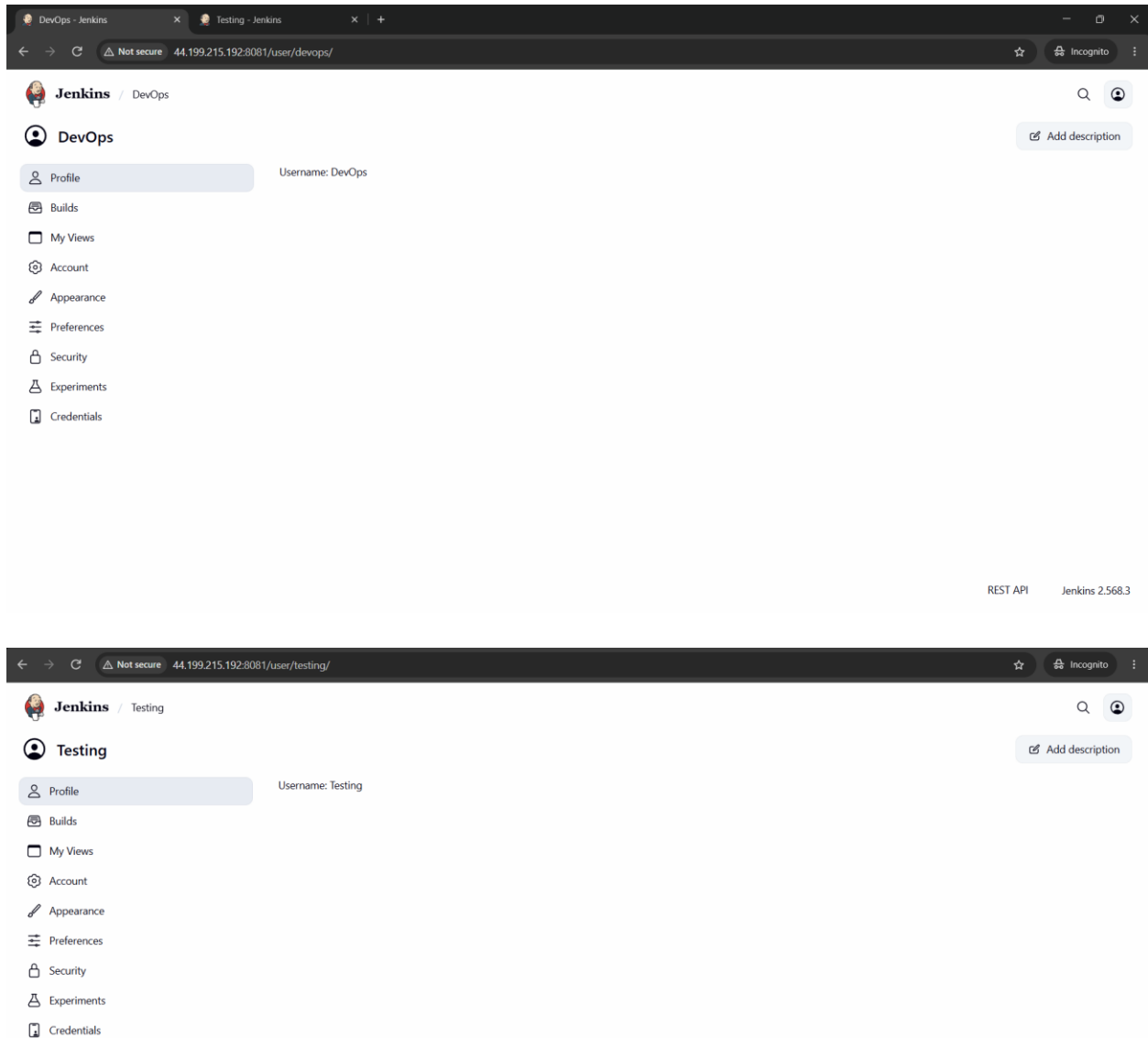
Overall → Read

Job → Read + Build

View → Read

This follows the principle of giving users only the permissions required for their work.





Result

The Jenkins user-management and authorization configuration was successfully completed.

Two additional Jenkins users were created:

- DevOps
- Testing

Matrix-based security was configured to provide controlled permissions to these users.

The administrator account asifshaik retained **Overall** → **Administer** permission.

Both DevOps and Testing were given limited permissions:

- Overall → Read
- Job → Read
- Job → Build
- View → Read

Neither user received administrator permissions.

The configuration was successfully tested by logging in as both users and confirming that **Manage Jenkins was not available** to them.

Conclusion

Jenkins Task 3 — **Create Users with Limited Access** was successfully completed.

The Jenkins server now has separate accounts for the administrator, DevOps user, and Testing user. Matrix-based security provides individual permission control instead of giving every authenticated user full administrative access.

The asifshaik account remains the Jenkins administrator, while DevOps and Testing have restricted access suitable for performing basic Jenkins job-related activities.

The successful login and access tests confirmed that the authorization configuration is working as expected.

Task 4 — Configure Labels and Restrict Jobs to Execute Based on Label Only

Objective

The objective of this task was to configure a Jenkins node with a specific label and restrict a Jenkins job so that it executes only on a node matching that label.

For this task, the Jenkins **Built-In Node** was assigned the label:

jenkins-node

A test Freestyle project named:

Label-Test-Job

was then configured to run only on nodes matching the jenkins-node label.

Step 1 — Open Jenkins Nodes

I logged in to Jenkins using the administrator account:

asifshaik

Then I navigated to:

Manage Jenkins → Nodes

Observation

The Nodes page displayed the Jenkins node:

Built-In Node

The node was initially showing as **offline**.

Step 2 — Investigate the Offline Node

I opened the **Built-In Node** to determine why it was offline.

The node displayed a disk-space warning:

Disk space is below threshold of 1.00 GiB.
Only 451.88 MiB out of 456.62 MiB left on /tmp.

Observation

The EC2 instance itself had sufficient disk space, but the /tmp filesystem was a small tmpfs.

I checked the /tmp filesystem using:

```
df -h /tmp
```

The output showed approximately:

Filesystem	Size	Used	Avail
tmpfs	457M	4.8M	452M

This showed that /tmp had only approximately **452 MB available**, which was below Jenkins' 1 GiB threshold.

Step 3 — Check /tmp Usage

I checked the contents of /tmp using:

```
sudo du -sh /tmp/*
```

Observation

The files inside /tmp were using only a small amount of space.

For example, the Jenkins-related temporary files were only a few MB or less.

Therefore, the problem was not excessive file usage. The issue was that /tmp itself was configured with a small maximum size.

Step 4 — Check /tmp Mount Configuration

I checked how /tmp was mounted:

```
mount | grep ' /tmp '
```

The output showed:

```
tmpfs on /tmp type tmpfs ... size=467580k
```

I also checked the systemd mount service:

```
systemctl status tmp.mount --no-pager
```

Observation

/tmp was configured as a **tmpfs** filesystem.

The tmp.mount service was active and /tmp was mounted as a temporary filesystem.

Step 5 — Configure a Larger /tmp

I created a systemd override for tmp.mount using:

```
systemctl edit tmp.mount
```

The following configuration was added:

```
[Mount]
Options=mode=1777,size=2G
```

This configured /tmp with a maximum size of **2 GB**.

Step 6 — Reload Systemd Configuration

I reloaded the systemd configuration:

```
systemctl daemon-reload
```

I initially attempted to restart the mount while Jenkins was running:

```
systemctl restart tmp.mount
```

The operation failed because /tmp was busy.

The system log showed:

umount: /tmp: target is busy

Observation

The /tmp filesystem could not be unmounted while processes were still using it.

Step 7 — Stop Jenkins Temporarily

To safely remount /tmp, I stopped Jenkins temporarily:

```
systemctl stop jenkins
```

Observation

Jenkins was stopped temporarily.

No Jenkins configuration or installation was removed.

Step 8 — Verify Processes Using /tmp

I checked whether any processes were still using /tmp:

```
fuser -vm /tmp
```

I also checked with:

```
lsof +D /tmp 2>/dev/null | head -30
```

Observation

There were no significant user processes holding /tmp.

The fuser output only showed the filesystem mount itself.

Step 9 — Remount /tmp

After stopping Jenkins and confirming that no user process was holding /tmp, I restarted the mount:

```
systemctl restart tmp.mount
```

Then I verified the filesystem:

```
df -h /tmp
```

Observation

The /tmp filesystem was successfully increased:

Filesystem	Size	Used	Avail
tmpfs	2.0G	0	2.0G

Result

The /tmp filesystem now had **2 GB available**, which was above Jenkins' 1 GiB disk-space threshold.

Step 10 — Start Jenkins Again

I started the Jenkins service:

```
systemctl start jenkins
```

I then verified the service:

```
systemctl status jenkins --no-pager
```

Observation

The Jenkins service was running successfully.

I also verified that Jenkins was still listening on port 8081:

```
ss -lntp | grep 8081
```

Result

Jenkins continued running on the configured port:

8081

Step 11 — Verify the Built-In Node

I returned to:

Manage Jenkins → Nodes

The **Built-In Node** was now online.

Observation

The previous /tmp disk-space warning was resolved, and the Built-In Node became available for executing Jenkins jobs.

Step 12 — Open Built-In Node Configuration

I opened:

Manage Jenkins → Nodes → Built-In Node → Configure

I located the:

Labels

field.

Step 13 — Assign a Label to the Built-In Node

I entered the following label:

jenkins-node

The label was then saved.

Observation

The Built-In Node was now associated with the label:

jenkins-node

This label can be used by Jenkins jobs to identify which node should execute a build.

Step 14 — Create a Test Jenkins Job

I returned to the Jenkins Dashboard and selected:

New Item

I created a Freestyle project named:

Label-Test-Job

Observation

The new Jenkins Freestyle project was successfully created.

Step 15 — Restrict the Job to the Label

I opened:

Label-Test-Job → Configure

Under **General**, I enabled:

Restrict where this project can be run

A **Label Expression** field appeared.

I entered:

jenkins-node

Configuration

Restrict where this project can be run: Enabled

Label Expression:

jenkins-node

Observation

The job was configured to execute only on a Jenkins node matching the jenkins-node label.

Step 16 — Add a Build Step

Under **Build Steps**, I selected:

Add build step → **Execute shell**

I entered:

```
echo "Running on Jenkins node with label: jenkins-node"  
hostname
```

Purpose

The first command provides clear evidence that the label-based test is being executed.

The hostname command identifies the actual machine on which the Jenkins build is running.

Step 17 — Save the Job Configuration

I clicked:

Save

The job configuration was successfully saved.

Observation

The job now had a node restriction based on the jenkins-node label.

Step 18 — Build the Job

I opened:

Label-Test-Job

and clicked:

Build Now

Jenkins created Build #1.

Step 19 — Verify Console Output

I opened:

Build #1 → Console Output

The console output showed:

Running on Jenkins node with label: jenkins-node

It also showed the hostname:

ip-172-31-11-210.ec2.internal

Finally, the build completed with:

Finished: SUCCESS

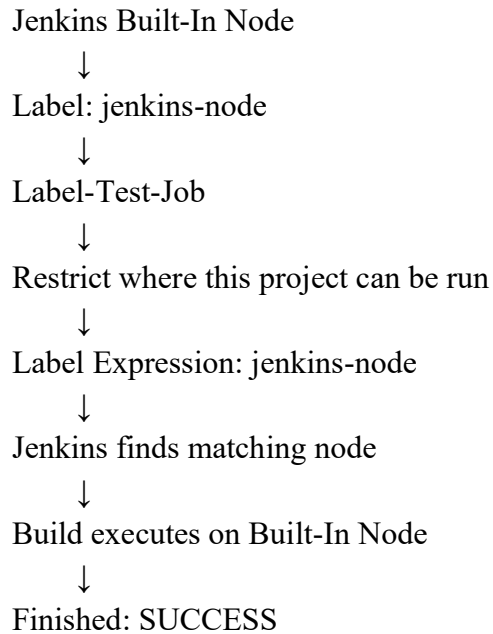
Observation

The build was successfully executed on the EC2 instance hosting the Jenkins Built-In Node.

The successful build demonstrated that Jenkins was able to find and execute the job on the node matching the configured label.

Label-Based Execution Flow

The final configuration can be represented as:



Configuration Summary

Component	Configuration
Jenkins Node	Built-In Node
Node Status	Online
Node Label	jenkins-node
Jenkins Job	Label-Test-Job
Job Type	Freestyle project
Restrict where project can be run	Enabled
Label Expression	jenkins-node
Build Step	Execute shell
Build Result	SUCCESS
Executing Host	ip-172-31-11-210.ec2.internal

Jenkins

Nodes

Built-In Node

Status

Configure

Build History

Load Statistics

Script Console

Build Executor Status

(0 of 2 executors busy)

Built-In Node

Mark this node temporarily offline

This is the Jenkins controller's built-in node. Builds running on this node will execute on the same system and as the same user as the Jenkins controller. This is appropriate e.g. for special jobs performing backups, but in general you should run builds on agents. [Learn more about distributed builds.](#)

Monitoring Data

Labels

jenkins-node

Projects tied to Built-In Node

None

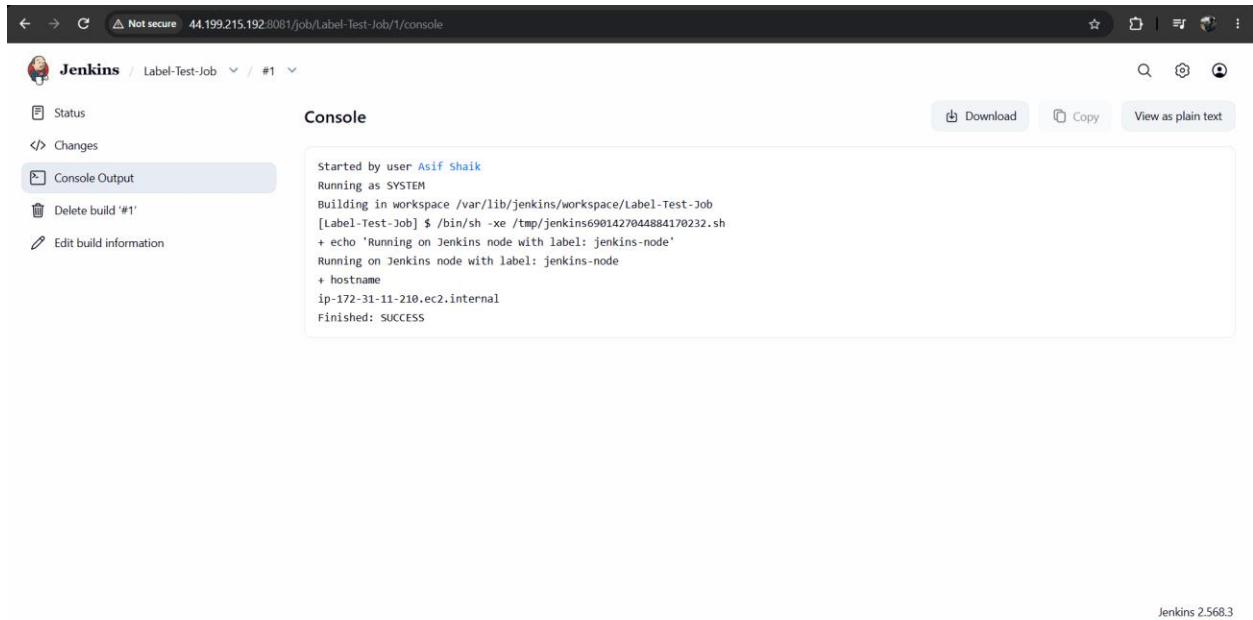
REST API

Jenkins 2.568.3

```
[root@ip-172-31-11-210 ec2-user]# systemctl status jenkins --no-pager
o Jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Drop-In: /etc/systemd/system/jenkins.service.d
            └─override.conf
   Active: inactive (dead) since Mon 2026-09-28 12:13:41 UTC; 7s ago
   Duration: 1h 57min 11.964s
   Process: 26954 ExecStart=/usr/bin/jenkins (code=exited, status=143)
   Main PID: 26954 (code=exited, status=143)
   Status: "jenkins stopped"
   CPU: 1min 28.237s

Sep 28 12:13:41 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 12:13:41.448+0000 [id=26] INFO h.p.b.global.Lifecycle#shutdown: Shutdown complete - Global Timeout Schedule.0 tasks pending
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 12:13:41.465+0000 [id=26] INFO jenkins.model.Jenkins!$onAttained: Completed termination
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 12:13:41.465+0000 [id=26] INFO jenkins.model.Jenkins#_cleanupDisconnectComputers: Starting node disconnection
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 12:13:41.472+0000 [id=26] INFO jenkins.model.Jenkins#_cleanupShutdownPluginManager: Stopping plugin manager
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 12:13:41.475+0000 [id=26] INFO jenkins.model.Jenkins#_cleanupPersistQueue: Persisting build queue
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 12:13:41.486+0000 [id=26] INFO jenkins.model.Jenkins#_cleanupWaitDisconnects: Waiting for node disconnection completion
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal jenkins[26954]: 2026-09-28 12:13:41.486+0000 [id=26] INFO hudson.lifecycle.Lifecycle!$onStatusUpdate: Jenkins stopped
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal systemd[1]: jenkins.service: Deactivated successfully.
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal systemd[1]: Stopped Jenkins.service - Jenkins Continuous Integration Server.
Sep 28 12:13:41 ip-172-31-11-210.ec2.internal systemd[1]: jenkins.service: Consumed 1min 28.237s CPU time.
Hint: Some lines were ellipsized, use -l to show in full.
[root@ip-172-31-11-210 ec2-user]# fusefs -vm /tmp
USER PID ACCESS COMMAND
root kernel mount /tmp
[root@ip-172-31-11-210 ec2-user]# lsuf -d /tmp 2>/dev/null | head -30
[root@ip-172-31-11-210 ec2-user]# systemctl restart tmp.mount
[root@ip-172-31-11-210 ec2-user]# df -h /tmp
Filesystem Size Used Avail Use% Mounted on
tmpfs 2.0G 0 2.0G 0% /tmp
[root@ip-172-31-11-210 ec2-user]# systemctl start jenkins
[root@ip-172-31-11-210 ec2-user]# systemctl status jenkins --no-pager
* Jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Drop-In: /etc/systemd/system/jenkins.service.d
            └─override.conf
   Active: active (running) since Mon 2026-09-28 12:16:08 UTC; 3s ago
   Main PID: 31266 (java)
   Tasks: 39 (limit: 1059)
   Memory: 328.4M
   CPU: 20.771s
   CGroup: /system.slice/jenkins.service
           └─31266 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8081

Sep 28 12:16:06 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:06.372+0000 [id=33] INFO jenkins.InitReactorRunner!$onAttained: Started all plugins
Sep 28 12:16:06 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:06.377+0000 [id=33] INFO jenkins.InitReactorRunner!$onAttained: Augmented all extensions
Sep 28 12:16:06 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:06.881+0000 [id=30] INFO h.p.b.g.GlobalTimeoutConfiguration!load: global timeout not set
Sep 28 12:16:08 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:08.502+0000 [id=31] INFO jenkins.InitReactorRunner!$onAttained: System config loaded
Sep 28 12:16:08 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:08.503+0000 [id=31] INFO jenkins.InitReactorRunner!$onAttained: System config adapted
Sep 28 12:16:08 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:08.504+0000 [id=32] INFO jenkins.InitReactorRunner!$onAttained: Loaded all jobs
Sep 28 12:16:08 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:08.520+0000 [id=32] INFO jenkins.InitReactorRunner!$onAttained: Configuration for all jobs updated
Sep 28 12:16:08 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:08.611+0000 [id=31] INFO jenkins.InitReactorRunner!$onAttained: Completed initialization
Sep 28 12:16:08 ip-172-31-11-210.ec2.internal jenkins[31266]: 2026-09-28 12:16:08.678+0000 [id=24] INFO hudson.lifecycle.Lifecycle!$onReady: Jenkins is fully up and running
Sep 28 12:16:08 ip-172-31-11-210.ec2.internal systemd[1]: Started Jenkins.service - Jenkins Continuous Integration Server.
[root@ip-172-31-11-210 ec2-user]# ss -ntlp | grep 8081
LISTEN 0 128 0.0.0.0:8081 0.0.0.0:* users:(("java",pid=31266,fd=9))
```

The screenshot shows the Jenkins web interface. The top navigation bar includes the Jenkins logo, the job name 'Label-Test-Job', and the build number '#1'. The left sidebar contains links for 'Status', 'Changes', 'Console Output' (which is selected), 'Delete build #1', and 'Edit build information'. The main area is titled 'Console' and displays the following output:

```
Started by user Asif Shaik
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Label-Test-Job
[Label-Test-Job] $ /bin/sh -xe /tmp/jenkins6901427044884170232.sh
+ echo 'Running on Jenkins node with label: jenkins-node'
Running on Jenkins node with label: jenkins-node
+ hostname
ip-172-31-11-210.ec2.internal
Finished: SUCCESS
```

On the right side of the console output, there are buttons for 'Download', 'Copy', and 'View as plain text'. The bottom right corner of the page indicates 'Jenkins 2.568.3'.

Result

The Jenkins Built-In Node was successfully brought online after resolving the /tmp disk-space threshold issue.

The node was assigned the label:

jenkins-node

A Freestyle project named Label-Test-Job was created and configured with:

Restrict where this project can be run → jenkins-node

The job was successfully executed, and the console output confirmed the label message and the EC2 hostname.

The build completed successfully with:

Finished: SUCCESS

Conclusion

Jenkins Task 4 — **Configure Labels and Restrict Jobs to Execute Based on Label** was successfully completed.

A label named jenkins-node was assigned to the Built-In Node, and the Label-Test-Job project was restricted to execute only on a node matching that label.

The successful Build #1 and its console output demonstrated that Jenkins scheduled the job on the matching Built-In Node.

This task provided practical understanding of **Jenkins nodes, labels, label expressions, and job-to-node assignment**. Labels allow Jenkins administrators to control where particular jobs are executed, which becomes especially useful when a Jenkins environment contains multiple build nodes with different capabilities.

Task 5 — Create Three sample jobs using the below URL.

- **Source Code:** https://github.com/betawins/Techie_horizon_Login_app.git

Objective

To create three sample Jenkins Freestyle jobs using the provided GitHub repository as the source code and configure all three jobs to execute only on the Jenkins node having the label jenkins-node.

Source Code Repository:

https://github.com/betawins/Techie_horizon_Login_app.git

Git Branch:

master

Jenkins Node Label:

jenkins-node

Job 1 — asifshaik-Job-1

Steps Used

1. Opened the Jenkins Dashboard.
2. Selected **New Item**.
3. Created a new **Freestyle project** named:

asifshaik-Job-1

4. Under **General**, enabled:

Restrict where this project can be run

5. Entered the label expression:

jenkins-node

6. Under **Source Code Management**, selected **Git**.
7. Configured the repository:

https://github.com/betawins/Techie_horizon_Login_app.git

8. Credentials were left as **None** because the repository is publicly accessible.

9. Configured the branch:

`*/master`

10. Added an **Execute shell** build step:

```
echo "Starting asifshaik-Job-1"  
echo "Cloning Techie Horizon Login App"  
pwd  
ls -la
```

11. Saved the job.

12. Selected **Build Now**.

13. Opened **Build #1** → **Console Output**.

Observation

The first build initially failed because `*/main` was configured while the repository uses the master branch.

After changing the branch to `*/master`, Jenkins successfully:

- Connected to the GitHub repository.
- Fetched the source code.
- Checked out the master branch.
- Created the Jenkins workspace.
- Executed the shell commands.
- Displayed the application files.

The build completed with:

Finished: SUCCESS

Job 2 — asifshaik-Job-2

Steps Used

1. Created a new **Freestyle project** named:

asifshaik-Job-2

2. Enabled:

Restrict where this project can be run

3. Entered:

jenkins-node

4. Selected **Git** under Source Code Management.
5. Configured the repository:

https://github.com/betawins/Techie_horizon_Login_app.git

6. Credentials were configured as:

None

7. Configured the branch:

*/master

8. Added an **Execute shell** build step:

```
echo "Starting asifshaik-Job-2"  
echo "Source code checkout successful"  
echo "Running on:"  
hostname  
echo "Workspace:"  
pwd
```

9. Saved the job.
10. Triggered the job using **Build Now**.
11. Checked the Console Output.

Observation

Jenkins used the configured Git repository and master branch. The job was restricted to the node with the jenkins-node label and executed the configured shell commands.

Job 3 — asifshaik-Job-3

Steps Used

1. Created a new **Freestyle project** named:

asifshaik-Job-3

2. Enabled:

Restrict where this project can be run

3. Entered the label:

jenkins-node

4. Selected **Git** under Source Code Management.
5. Configured the repository:

https://github.com/betawins/Techie_horizon_Login_app.git

6. Credentials were configured as:

None

7. Configured the branch:

*/master

8. Added an **Execute shell** build step:

```
echo "Starting asifshaik-Job-3"  
echo "Techie Horizon Login App"  
echo "Running on node:"  
hostname  
echo "Build completed successfully"
```

9. Saved the job.
10. Triggered the job using **Build Now**.
11. Checked the Console Output.

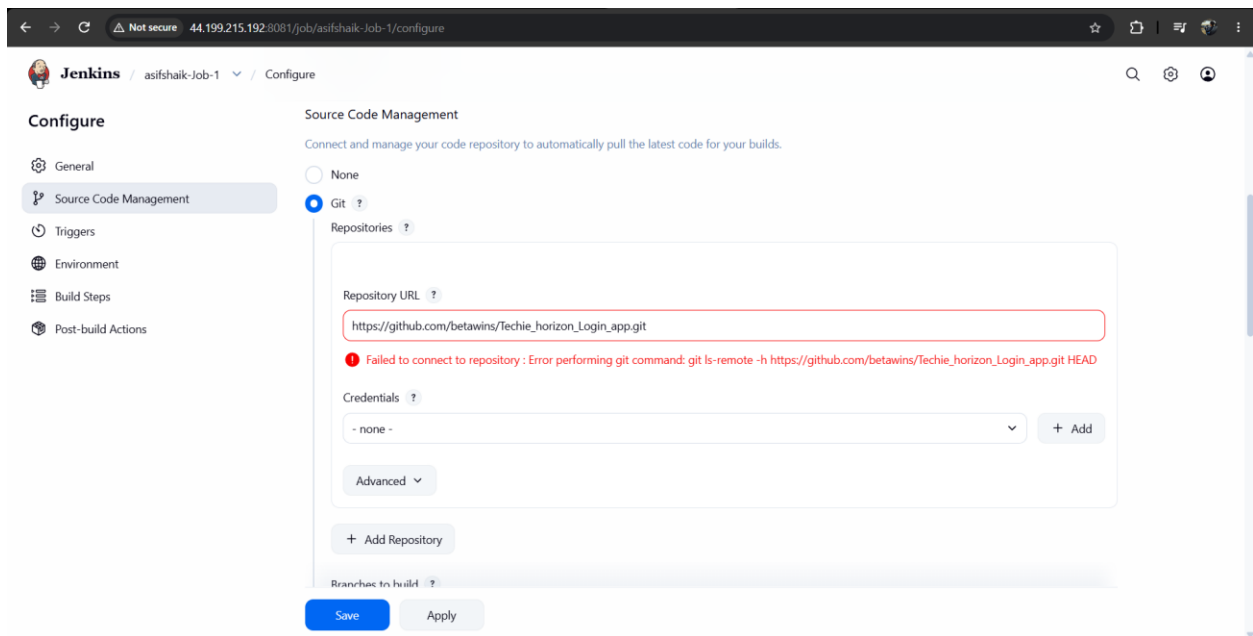
Observation

Jenkins was configured to retrieve the source code from the public GitHub repository using the master branch. The job was restricted to the jenkins-node labeled node.

Overall Result

Three sample Jenkins Freestyle jobs were created:

Job	Type	Git Repository	Branch	Node Label
asifshaik-Job-1	Freestyle	Techie_horizon_Login_app	master	jenkins-node
asifshaik-Job-2	Freestyle	Techie_horizon_Login_app	master	jenkins-node
asifshaik-Job-3	Freestyle	Techie_horizon_Login_app	master	jenkins-node



```
[root@ip-172-31-11-210 ec2-user]# git --version
git version 2.50.1
```

←

→

↺

⚠ Not secure

44.199.215.192:8081/job/asisfshaik-Job-1/configure

☆

🔍

⚙

👤

 **Jenkins**

asisfshaik-Job-1 / Configure

🔍

⚙

👤

Configure

⚙ General

🔑 Source Code Management

🕒 Triggers

🌐 Environment

📋 Build Steps

🔗 Post-build Actions

☒ Restrict where this project can be run ?

Label Expression ?
jenkins-node
Label jenkins-node matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

Advanced ▾

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?
https://github.com/betawins/Techie_horizon_Login_app.git

Credentials ?

Save

Apply

← → ↻ Not secure 44.199.215.192:8081/job/asifshaik-Job-1/configure

Jenkins / asifshaik-Job-1 / Configure

Configure

- General
- Source Code Management
- Triggers
- Environment**
- Build Steps
- Post-build Actions

☐ Terminate a build if it's stuck

☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

See the list of available environment variables

```
echo "Starting asifshaik-Job-1"
echo "Cloning Techie Horizon Login App"
pwd
ls -la
```

Advanced ▾

+ Add build step

Save Apply

← → ↻ Not secure 44.199.215.192:8081/job/asifshaik-Job-1/2/console

Jenkins / asifshaik-Job-1 / #2 ▾

```
> git rev-parse refs/remotes/origin/master^[commit] # timeout=10
Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Commit message: "sabit code for testing"
First time build. Skipping changelog.
[asifshaik-Job-1] $ /bin/sh -xe /tmp/jenkins331026862324628230.sh
+ echo 'Starting asifshaik-Job-1'
Starting asifshaik-Job-1
+ echo 'Cloning Techie Horizon Login App'
Cloning Techie Horizon Login App
+ pwd
/var/lib/jenkins/workspace/asifshaik-Job-1
+ ls -la
total 20
drwxr-xr-x. 6 jenkins jenkins 136 Sep 28 12:47 .
drwxr-xr-x. 4 jenkins jenkins 51 Sep 28 12:44 ..
drwxr-xr-x. 7 jenkins jenkins 146 Sep 28 12:47 .git
drwxr-xr-x. 3 jenkins jenkins 23 Sep 28 12:47 .github
-rw-r--r--. 1 jenkins jenkins 137 Sep 28 12:47 Dockerfile
-rw-r--r--. 1 jenkins jenkins 978 Sep 28 12:47 README.md
drwxr-xr-x. 2 jenkins jenkins 91 Sep 28 12:47 ansible
-rw-r--r--. 1 jenkins jenkins 508 Sep 28 12:47 azure-pipelines.yml
-rw-r--r--. 1 jenkins jenkins 5830 Sep 28 12:47 pom.xml
drwxr-xr-x. 4 jenkins jenkins 30 Sep 28 12:47 src
Finished: SUCCESS
```

← → ↻ Not secure 44.199.215.192:8081/job/asifshaik-Job-2/configure

Jenkins / asifshaik-Job-2 / Configure

Configure

- General
- Source Code Management**
- Triggers
- Environment
- Build Steps
- Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

Credentials ?

+ Add

Advanced ▾

+ Add Repository

Branches to build ?

Save Apply

← → ↻ Not secure 44.199.215.192:8081/job/asifshaik-Job-2/1/console

Jenkins / asifshaik-Job-2 / #1

```
<div data-bbox="306 446 833 694" data-label="Text">

Receiving updates from repository https://github.com/betawins/Techie_horizon_login_app.git



> git --version # timeout=10



> git --version # 'git version 2.50.1'



> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_login_app.git +refs/heads/*:refs/remotes/origin/* # timeout=10



> git config remote.origin.url https://github.com/betawins/Techie_horizon_login_app.git # timeout=10



> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10



Avoid second fetch



> git rev-parse refs/remotes/origin/master^{commit} # timeout=10



Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)



> git config core.sparsecheckout # timeout=10



> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10



Commit message: "sabit code for testing"



First time build. Skipping changelog.



[asifshaik-Job-2] $ /bin/sh -xe /tmp/jenkins8494530355682383863.sh



+ echo 'Starting asifshaik-Job-2'



Starting asifshaik-Job-2



+ echo 'Source code checkout successful'



Source code checkout successful



+ echo 'Running on:'



Running on:



+ hostname



ip-172-31-11-210.ec2.internal



+ echo Workspace:



Workspace:



+ pwd



/var/lib/jenkins/workspace/asifshaik-Job-2



Finished: SUCCESS


```

← → ↻ Not secure 44.199.215.192:8081/job/asisfshaik-Job-3/configure

Jenkins / asifshaik-Job-3 / Configure

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/betawins/Techie_horizon_Login_app.git

Credentials ?

- none - + Add

Advanced ▾

+ Add Repository

Branches to build ?

Save Apply

← → ↻ Not secure 44.199.215.192:8081/job/asisfshaik-Job-3/1/console

Jenkins / asifshaik-Job-3 / #1

CLAUUUU IIRUUUUUUUU

```
Cloning repository https://github.com/betawins/Techie_horizon_Login_app.git
> git init /var/lib/jenkins/workspace/asisfshaik-Job-3 # timeout=10
Fetching upstream changes from https://github.com/betawins/Techie_horizon_Login_app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_Login_app.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git config remote.origin.url https://github.com/betawins/Techie_horizon_Login_app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Commit message: "sabir code for testing"
First time build. Skipping changelog.
[asisfshaik-Job-3] $ /bin/sh -xe /tmp/jenkins14507133166811337543.sh
+ echo 'Starting asifshaik-Job-3'
Starting asifshaik-Job-3
+ echo 'Techie Horizon Login App'
Techie Horizon Login App
+ echo 'Running on node:'
Running on node:
+ hostname
ip-172-31-11-210.ec2.internal
+ echo 'Build completed successfully'
Build completed successfully
Finished: SUCCESS
```

The screenshot shows the Jenkins web interface in a browser. The top navigation bar includes the Jenkins logo, a 'New Item' button, and a 'Build History' link. On the right, there are search, settings, and user profile icons, along with an 'Add description' button. The left sidebar contains a 'Build Queue' section (showing 'No builds in the queue.') and a 'Build Executor Status' section (showing '0 of 2 executors busy'). The main content area displays a table of build history with columns for status, icon, name, last success, last failure, and last duration. There are four builds listed, all with a success status and a sun icon. Below the table is a horizontal scrollbar and a legend for the 'Icon: S M L' categories. The bottom right corner shows 'REST API' and 'Jenkins 2.568.3'.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	asifshaik-Job-1	7 min 32 sec #2	N/A	3.3 sec
✓	☀	asifshaik-Job-2	3 min 17 sec #1	N/A	0.36 sec
✓	☀	asifshaik-Job-3	51 sec #1	N/A	0.96 sec
✓	☀	Label-Test-Job	30 min #1	N/A	86 ms

Result

The three sample Jenkins jobs were configured to use the provided GitHub repository and execute on the Jenkins node matching the jenkins-node label.

Conclusion

Task 5 was completed by creating three sample Jenkins Freestyle jobs. Each job uses the same GitHub source repository, checks out the master branch, and is restricted to execute on the node with the jenkins-node label.

This demonstrates Jenkins integration with Git, source-code checkout, Freestyle job configuration, and label-based job execution.

Task 6 — Create One Jenkins Job Using GitHub Private Repository

Objective

To create a Jenkins Freestyle job that connects to a **private GitHub repository** using secure Jenkins credentials, checks out the main branch, and executes a build successfully.

Private GitHub Repository:

<https://github.com/asifshaik6558/jenkins-private-repo.git>

Jenkins Job:

asifshaik-Private-Repo-Job

Git Branch:

main

Step 1 — Create a Private GitHub Repository

A private GitHub repository named:

jenkins-private-repo

was created.

The repository visibility was set to:

Private

Repository:

<https://github.com/asifshaik6558/jenkins-private-repo.git>

Initially, the repository was empty.

Step 2 — Create GitHub Personal Access Token

A GitHub Personal Access Token was created to allow Jenkins to authenticate with the private repository.

The token was configured with the required repository access.

The token was **not exposed in the Jenkins job configuration**. It was stored securely as a Jenkins credential.

Step 3 — Add GitHub Credentials to Jenkins

Navigated to:

Jenkins → Manage Jenkins → Credentials → System → Global credentials → Add Credentials

Configured the credential as:

Kind:

Username with password

Username:

asifshaik6558

Password:

GitHub Personal Access Token

ID:

github-private-repo

Description:

GitHub credentials for private repository

The credential was successfully created.

Step 4 — Create Jenkins Job

Navigated to:

Jenkins Dashboard → New Item

Created a new:

asifshaik-Private-Repo-Job

Selected:

Freestyle project

Step 5 — Configure Node Label

Under **General**, enabled:

Restrict where this project can be run

Configured the label expression:

jenkins-node

This ensures the job executes only on the Jenkins node configured with the jenkins-node label.

Step 6 — Configure Private Git Repository

Under **Source Code Management**, selected:

Git

Configured the repository URL:

<https://github.com/asifshaik6558/jenkins-private-repo.git>

Selected the Jenkins credential:

github-private-repo

Configured the branch:

*/main

Step 7 — Configure Build Step

Under **Build Steps**, selected:

Add build step → **Execute shell**

The following commands were configured:

```
echo "Private GitHub repository checkout successful"
echo "Running on Jenkins node:"
hostname
echo "Workspace:"
pwd
ls -la
```

The job configuration was saved.

Step 8 — First Build Observation

The first build initially failed with:

```
ERROR: Couldn't find any revision to build.
Verify the repository and branch configuration for this job.
```

Reason

The Jenkins credential and Git authentication were working correctly, but the private GitHub repository was initially empty and did not contain a commit on the main branch.

The console confirmed that Jenkins was using the configured credential:

```
using credential github-private-repo
```

Therefore, the issue was with the repository having no revision to check out, not with the Jenkins credential.

Step 9 — Add Initial Commit to GitHub

A README.md file was created in the private GitHub repository.

Content was added and committed to:

main

This created the first commit in the repository.

Step 10 — Run Jenkins Job Again

The Jenkins job was executed again using:

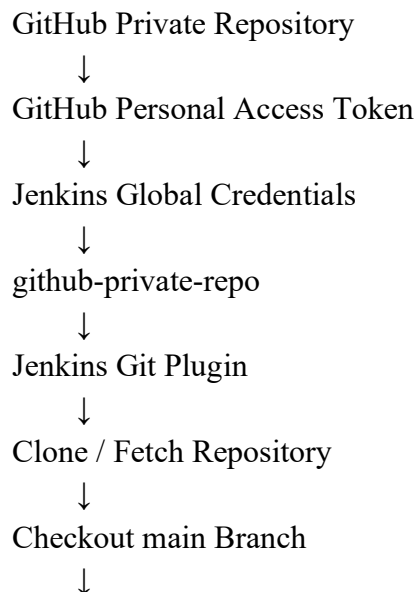
Build Now

Jenkins successfully authenticated with the private repository, fetched the repository, found the main branch, checked out the source code, and executed the configured shell commands.

The second build completed successfully:

Finished: SUCCESS

Flow



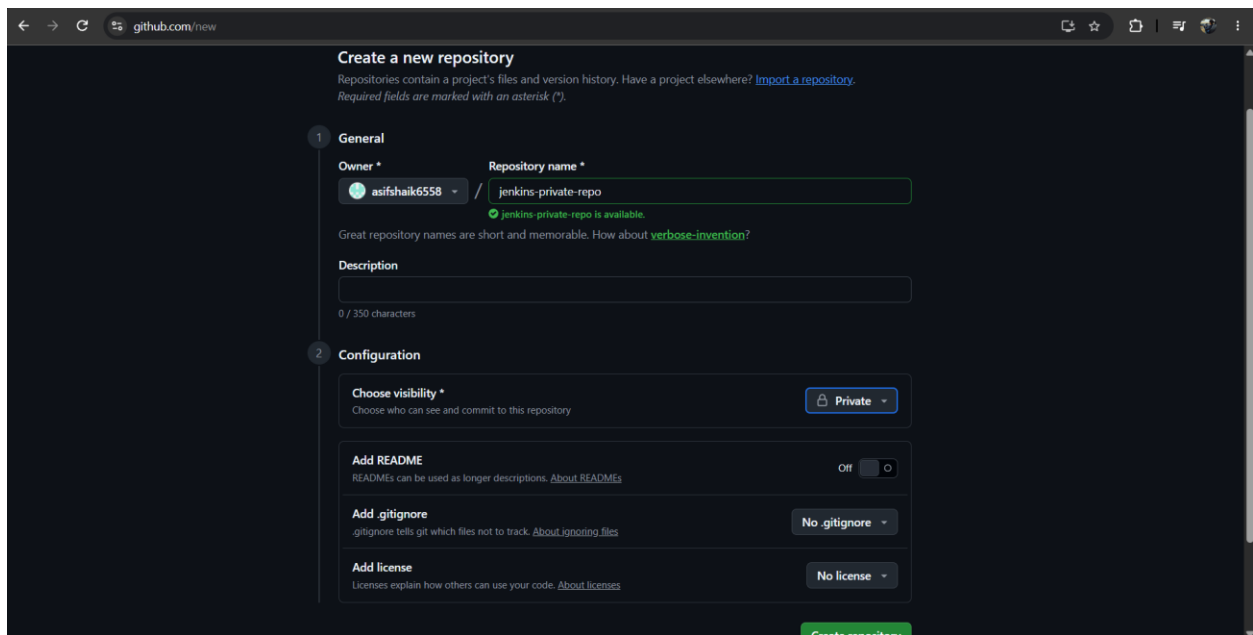
Execute Shell

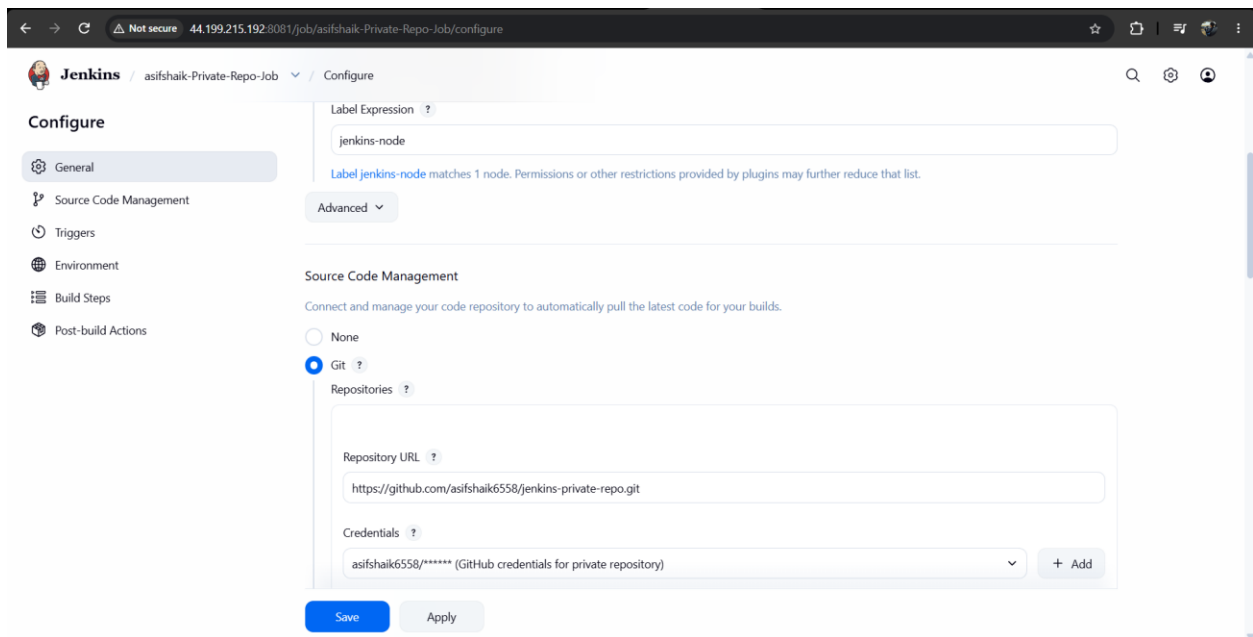
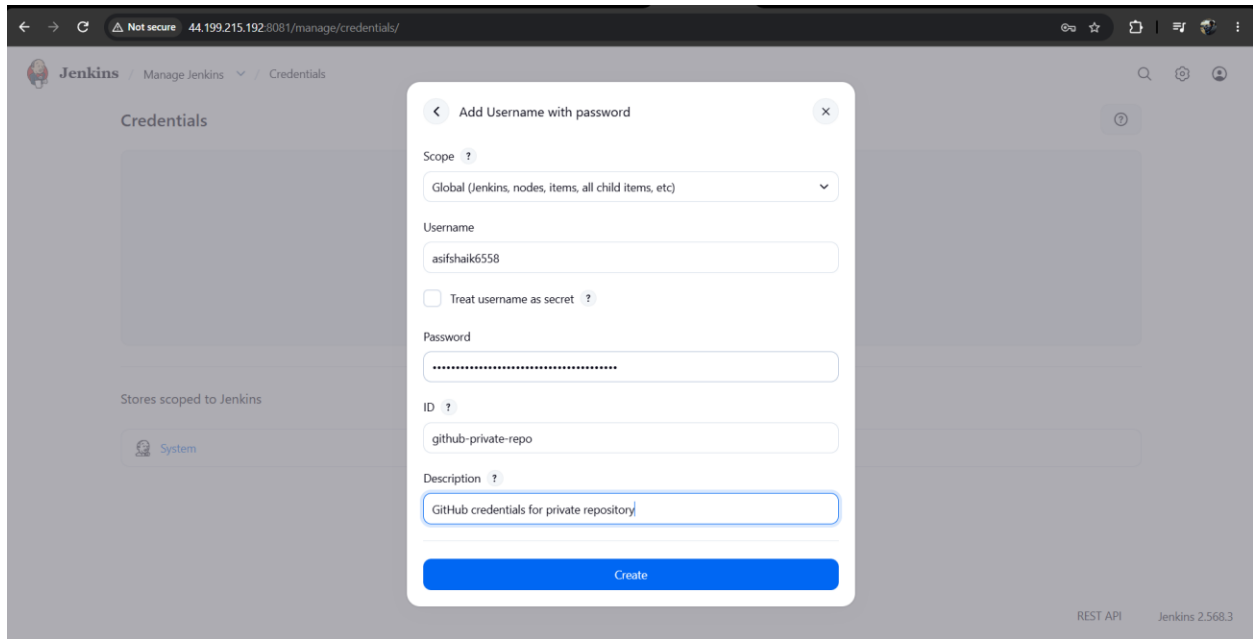


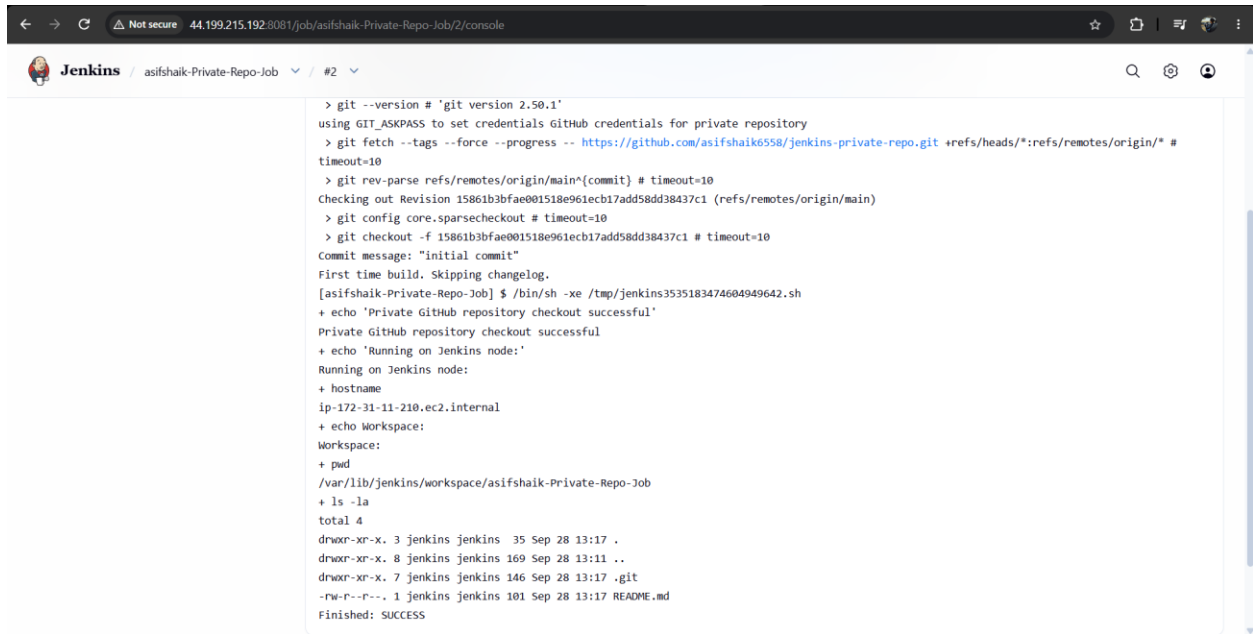
Build SUCCESS

Observation

- Jenkins successfully authenticated with the private GitHub repository.
- The GitHub Personal Access Token was stored as a Jenkins credential instead of being directly exposed in the job configuration.
- Jenkins successfully accessed the private repository after an initial commit was added.
- The main branch was successfully checked out.
- The job executed on the node with the jenkins-node label.
- The build completed with SUCCESS.





A screenshot of a web browser displaying the Jenkins console output for a job named 'asifshaik-Private-Repo-Job'. The browser's address bar shows the URL '44.199.215.192:8081/job/asifshaik-Private-Repo-Job/2/console'. The Jenkins interface includes a search bar and a user profile icon. The console output shows a series of commands and their results, including setting up Git credentials, fetching the repository, checking out the main branch, and running a build script. The build script includes commands for setting the hostname, workspace, and running a shell script. The output concludes with a directory listing and a 'Finished: SUCCESS' message.

```
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials GitHub credentials for private repository
> git fetch --tags --force --progress -- https://github.com/asifshaik6558/jenkins-private-repo.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/main^(commit) # timeout=10
Checking out Revision 15861b3bfae001518e961ecb17add58dd38437c1 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 15861b3bfae001518e961ecb17add58dd38437c1 # timeout=10
Commit message: "initial commit"
First time build. Skipping changelog.
[asifshaik-Private-Repo-Job] $ /bin/sh -xe /tmp/jenkins3535183474604949642.sh
+ echo 'Private GitHub repository checkout successful'
Private GitHub repository checkout successful
+ echo 'Running on Jenkins node:'
Running on Jenkins node:
+ hostname
ip-172-31-11-210.ec2.internal
+ echo Workspace:
Workspace:
+ pwd
/var/lib/jenkins/workspace/asifshaik-Private-Repo-Job
+ ls -la
total 4
drwxr-xr-x. 3 jenkins jenkins 35 Sep 28 13:17 .
drwxr-xr-x. 8 jenkins jenkins 169 Sep 28 13:11 ..
drwxr-xr-x. 7 jenkins jenkins 146 Sep 28 13:17 .git
-rw-r--r--. 1 jenkins jenkins 101 Sep 28 13:17 README.md
Finished: SUCCESS
```

Result

The Jenkins Freestyle job:

asifshaik-Private-Repo-Job

successfully connected to the private GitHub repository using the Jenkins credential:

github-private-repo

and successfully checked out the main branch and executed the build.

Result: SUCCESS

Conclusion

Task 6 was successfully completed. A Jenkins Freestyle job was created to access a private GitHub repository using securely stored Jenkins credentials. The successful build demonstrated Jenkins authentication with a private repository, Git source-code checkout, label-based node execution, and build execution.